Name: Ioannis Drivas
AM: 5216

Data Analysis

Data analysis is an extremely important step to help us understand how to approach the given problem since we can get an idea of which data are relevant and important to use and which needs to be removed. We distinguished some of the patterns that our initial data had and we found three main categories:

1. Data with a specific structure with their respective subtitle(e.g Date Opened:).
2. Repeated paragraphs describing a recall.
3. Data containing html tags

This step helps us clarify where we need to focus when we preprocess the data so that we can remove most of the noise and keep the valuable information in order to achieve a better prediction.

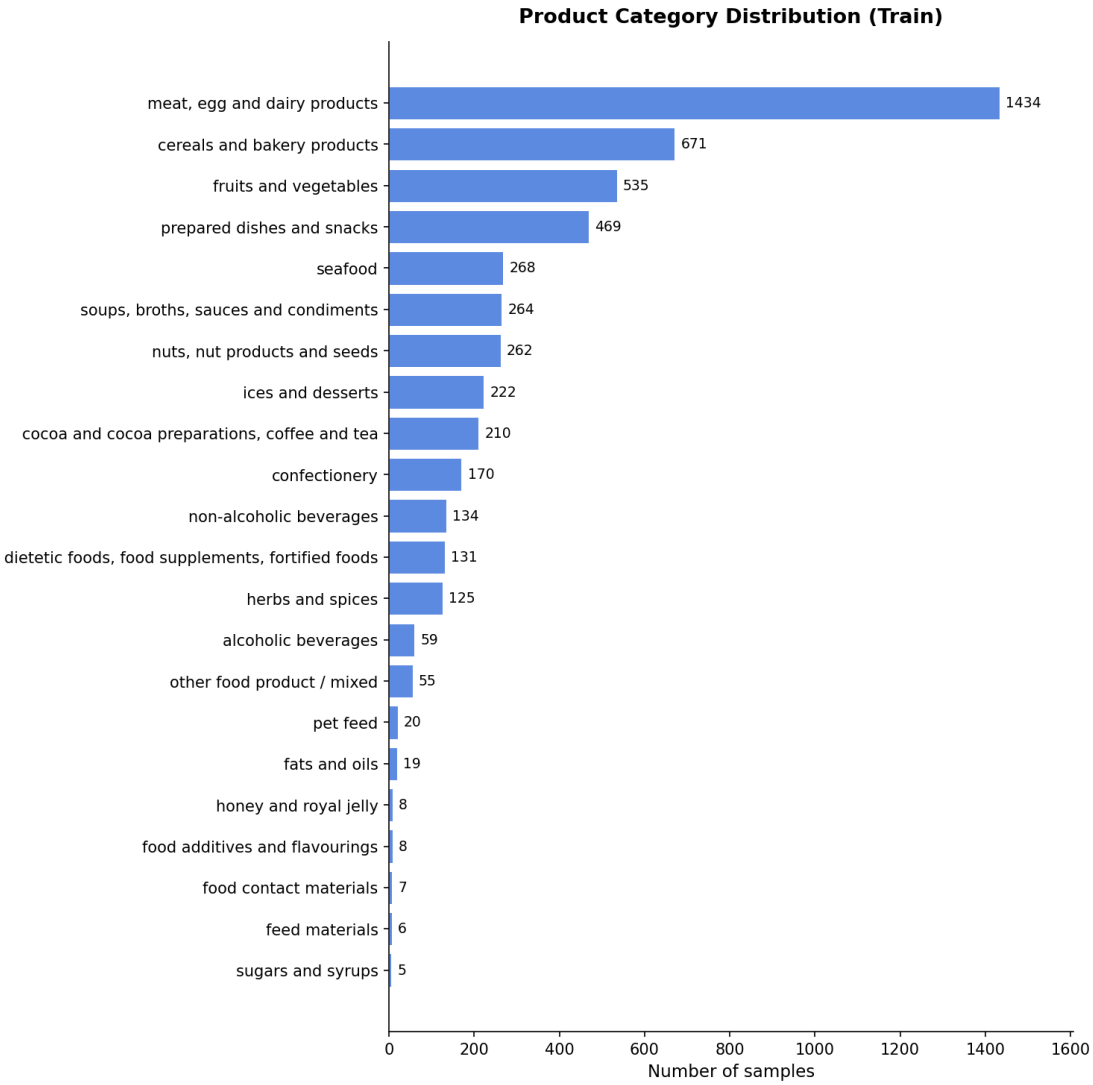
The next step was to locate imbalance on our categories as mentioned on the instructions of our task. We can clearly see through an example that a hazard-category of “packaging defect” appears a lot less than “allergens”. This observation is crucial because it will directly affect our model due to having minimal training data for one category and a plethora of training data on the other. This can lead to a lot of different paths on how we can overcome the problem with some ways being:

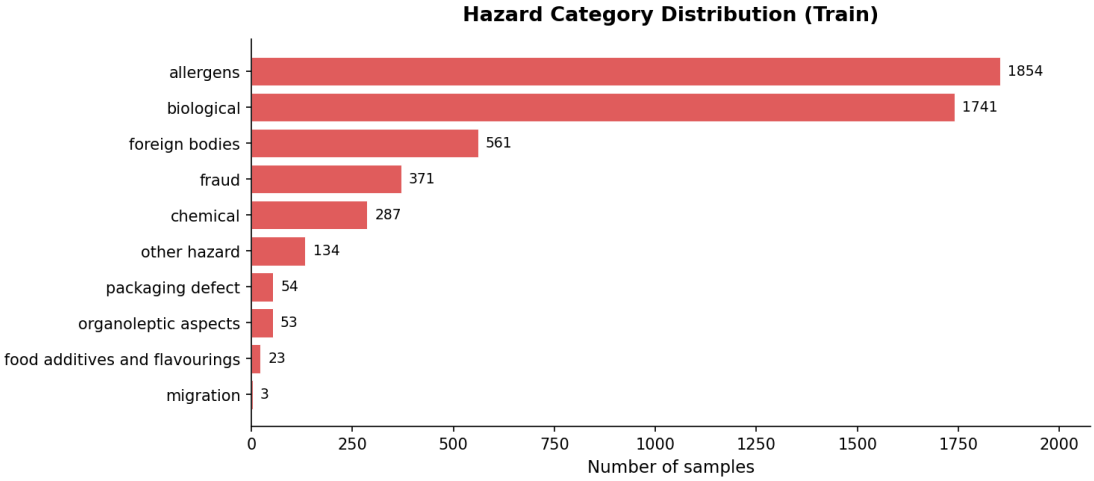
• Adding more training data to these categories
• Undersampling categories with more training data than others
• Higher penalization when guessing wrong classes that are shown less
• Having a confusion matrix when evaluating to find this imbalanced domination of one class over the other

We decided not to implement any of the above other than the confusion matrix since we already achieved a decent score with the baseline methods.

Finally we did not find any unusual characters or the use of another language other than English so we will not have to factor them in at the preprocessing stage.

Visualization of the categories





Preprocessing

The preprocessing stage is where the data analysis is being implemented on our data and start to go from observations to actions.

Our first step was to remove duplicate lines from our data which is achieved by iterating each line of our data and depending on if we have seen again a line or not we either discard it or add it to a list of unique lines.

Our second action is to convert our words to lower case since having upper case words would count as different instances and do not give as any additional information. We also targeted specific occurrences such as urls, numbers, emails and html tags because they are just noise.

Then we created a labels list and added some specific patterns we found throughout the data analysis stage. These patterns are words and phrases that seemed to be present on a lot of texts, so we captured most of them and removed them from our words list.

Next, we removed punctuations and special characters such as (\n, whitespaces).

At last, we used methods such as stemming and removing stop words. These two provide some way of dimensionality reduction. Stemming can fuse multiple words of the same family into one (e.g activation and activate can be written as active) where on the other hand stop words remove unnecessary words that are fillers with no meaningful information. Note that for the stop words we also created a custom stop word list of our own with words that we had found from our analysis and thought the imported library might not include them (e,g january). Finally, words such as: not etc, were not quarantined from our stop words because the format of the given text explicitly states the hazard and the product, so keeping it would not help our model with better predictions. We also discarded words with length <2, so characters that cannot provide us useful information.

Feature Extraction Document Term Matrix

For the baseline solution we used two different methods, the tf-idf and the word2vec.

The tf-idf method creates an array where each row represents a vector of a document and each column represents a vector of a word. In this way we can calculate for a given word, its frequency on each document and for a given document, we can calculate the frequency of each word in it. This method is called via the TfidfVectorizer from the library sklearn and we gave it us input the min_df = 5 where words that are present on less than 5 documents are excluded and max_df = 0.85 where words are present on more than 85% of the documents are excluded. Finally, we call the method fit_transform

Word2vec method is from the same library but utilizes the words in a different way. As parameters it takes the following:

• Sentences: is a list of all the words in a document
• Vector_size: the size of the vector that each word has
• Window: the number of words that will be used for skipgram
• Min_count: ignores words with lower frequency than what is set
• Sg : mode for skipgram or cbow
• Workers: number of threads to be utilized
• Seed: static value for consistency in test results since vectors are randomly initialized

Word2vec can be implemented using cbow or skipgram. Cbow (Continuous bag of words) is a method where it is using its surrounding words to predict the center word whereas skipgram uses the center word to predict the surroundings relative to the value of window. Based on the theory cbow is always worse than skipgram so we decided to not implement it at all.

Since word2vec is creating a vector for each word and a document contains a lot of them, after calling word2vec we take the mean and produce a final vector with size the value of vector_size through calling the method get_doc_vector. Note that to get the correct dimension of the final vector when we calculate the mean with the library numpy, we also need to specify axis =0.

At this stage we fit and transform our data to get them ready for our models. When we say fit, we mean the creation of the vocabulary whereas, transformation is the procedure where we turn the words into vectors and create the correct dimensions.

Dimensionality Reduction

We only used SVD on the tf-idf method from the sklearn library with parameters:

• n_components: the size of the final vector
• seed: static value for consistency in test results since vectors are randomly initialized

Since we have done a lot of preprocessing, reducing the dimensions will not give us any substantial benefit because there is not really any noise left inside our words.

Model Learning

For our baseline we used the following models from the sklearn library:

• MultinomialBayes
• ComplementBayes
• GaussianBayes
• Logistic Regression

MultinomialBayes assumes that each word is independent from all the others. It calculates the probability of each category and then it calculates the probability of the word based on the category that it is given. Then the final probability is the product of the prior probability of the category and the conditional probabilities of all the words found in the document.

ComplementBayes is the tuned version of the MultinomialBayes and is more fit for our problem since we have imbalanced categories. This method does not calculate the probability of the word based on the category that it is given but it calculates the probability of the word based on not being on all others.

GaussianBayes is used for the SVD and word2vec implementation and uses the mean and standard deviation of a word in a category. It is used on these two because these methods can produce negative values and only the GaussianBayes of the 3 above can interpret negative values.

Logistic Regression uses a sigmoid activation function and assigns weight to each word giving the probability of a word being in a specific category. Using as parameter the class_weight = balanced we can achieve what the above couldn't, fixing the category imbalance problem. We also set a max iteration for 2000 since we have a lot of words, and it states after how many iterations the algorithm will stop if it hasn't already decided the optimal settings.

All the above models use methods that fit and predict. At the model stage when we use fit, we essentially train our model based on the training data so it can learn how to distinguish which text corresponds to the correct category. When it comes to testing, we can only transform the testing data to create the vectors and then we use the predict method to let our model guess the correct category.

Fine Tuning

The use of optimal parameters can significantly improve our model and is a necessary step. Fine tuning is this specific procedure, and we implemented it for the logistic regression model. We used the gridSearchCV method from sklearn to achieve that giving it the following parameters:

- Base_lr: is the logistic regression as shown above
- Param_grid: a list of values that we want to find the best from them
- Cv: 5 is the default value and represents a 5-fold cross validation. This means that the training data are split into 5 parts and each time each part is a validation set that we see how many we got correct based on the training of the other four. On the next iteration we take the next part and do the same procedure.
- Scoring: we chose the scoring method of cross-validation. We used the f1 macro
- N_jobs: we set it to -1 so we use all the processing power.

For the logistic regression we gave a param_grid of C = [0.1, 1.0, 10.0, 100.0]. This parameter is a penalization factor. When its low our model is simpler and captures the "big picture" which helps from overfitting. Having higher value gives the model more freedom which helps take into consideration rarer words but has the risk of overfitting.

Text Categorization

At this stage we decided to implement a method called ensembling. Instead of running each method alone and picking the best of them we combined multiple methods together so each one can contribute its predictions and have a more spherical answer. We chose logistic regression and complement Bayes since both use the tf-idf. To add Multinomial Bayes would only hurt us since complement Bayes is already better and to add Gaussian Naïve Bayes, we would have to bring tf-idf and word2vec and svd into the same dimensions which seemed a little hard to do. We used the method VotingClassifier that takes as estimators the logistic regression and complement Bayes and as a voting parameter we set it to soft which takes the mean of the two probabilities of the predictions.

State of the Art Implementation

The last implementation we tried was a state-of-the-art method using a BERT model. The first step was to change the clean text function by cleaning only duplicate lines, urls, emails and html tags. We didn't use any stemming, stop

word removal, lower casing the words and removing numbers because the transformers need that information and by removing it, we worsen the model's learning ability. The next step is to add a focal loss function for our model. This is important due to the imbalance of our data. Its use is to reduce the weight of usual examples and reinforce the weight of rare classes such as (migration, honey and royal jelly, food additives and flavorings, sugar and syrups, feed materials). To balance the data, we also augmented the train set by choosing the above categories, finding examples from the train set and giving them to known LLMS and creating similar examples. We created 20 examples for each category and added them to the bottom of the train csv file.

Later we created the train_transformer method where we transform the categories into numbers and compute their weights. At this point we also used the clamp method from pytorch because some classes gave as "nan" values because we had no occurrences. Then we take the altered train and valid datasets and we tokenize them. Since most of the text contains a lot of words, we used a max_length of 512 but note that on weaker graphics cards, this size can give a memory error. After that we load our model and set its arguments. We specified the number of epochs, the batch size for the training and validation, the warmup_ratio which gradually bumps up the learning rate in order to have smooth steps at the beginning, the warmup_decay which applies a penalty when the model relies too heavily on a specific word forcing its attention to the whole sentence, eval/save_strategy= "epoch" which stores and evaluates the model against the validation set at the end of each epoch, load_best_model_at_end which loads the best checkpoint it found, learning_rate which is how quickly it learns from the data set, metric_for_best_model which specifies the evaluation function for the load_best_model_at_end, greater_is_better which specifies that the best f1 score achieved at an epoch is the optimal, save_total_limit which helps with the disk space occupied from saving each epoch, bf16 which uses 16 bit numbers instead of 32 helping with vram usage and matrix multiplications and logging_steps which prints the training loss for every given batch.

Finally, we pass our trainer through the Focal Loss function, begin the train and once it's finished, we start the prediction on the validation set and convert the numbers again to their category.

Evaluation

The creation of stats and evaluating them is important to find out what methods worked, where we missed some predictions and the overall general predictivity of our model.

First, we implemented the given equation using the f1 score. We gave it as parameters below:

• Y_true_hazard: the real hazards
• Y_pred_hazard: the predicted hazards
• Average: a macro f1 score
• Zero division: we set it to 0 so classes that haven't been predicted at all correspond to a score of 0 and don't give as an error

Then using the numpy library we create an array that has True/False based on if we predicted the answer correctly or not. From that we can find the predicted products categories based on correct predictions of hazard categories. Then we calculate the final score.

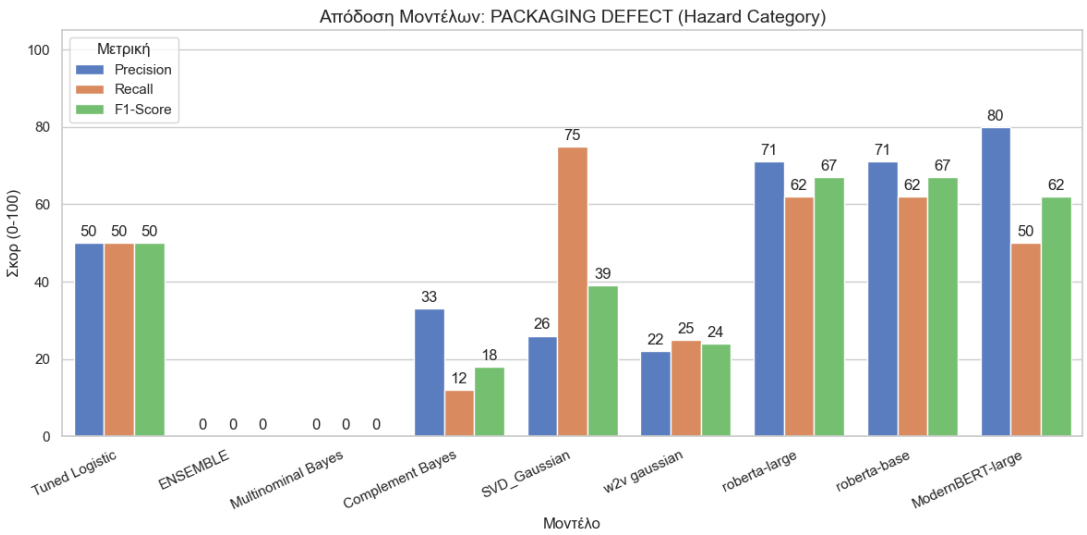
Then we used the method classification_report on both categories showing us the following:

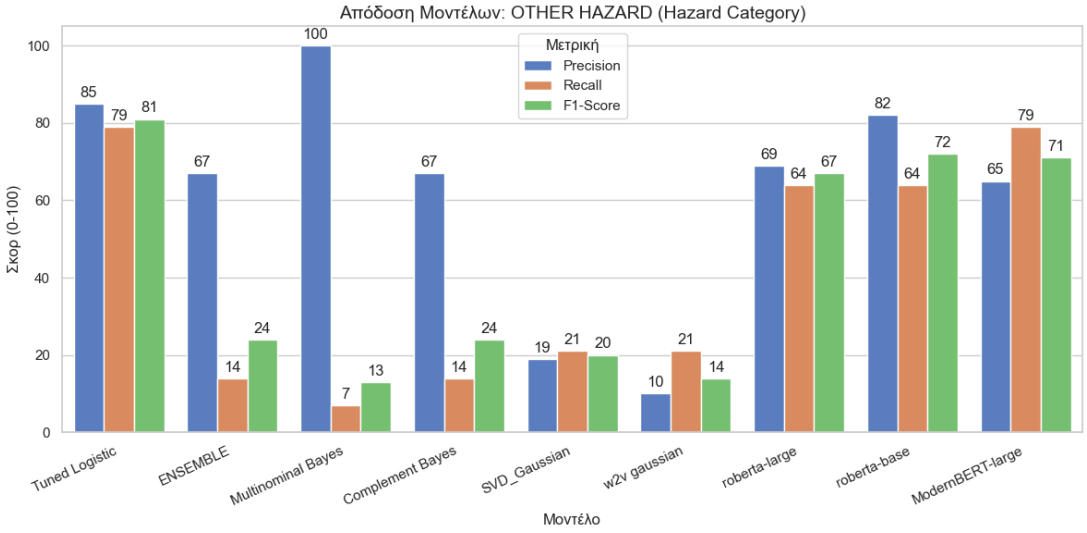
• Precision: the accuracy of correct predictions. Lower scores mean that we have false positives.
• Recall: how many correct answers we found from the data. Lower scores mean that we have false negatives.
• F1-score: Specialized mean of precision and recall. High imbalances between precision and recall provide a low score.
• Support: How many true samples were inside the data.

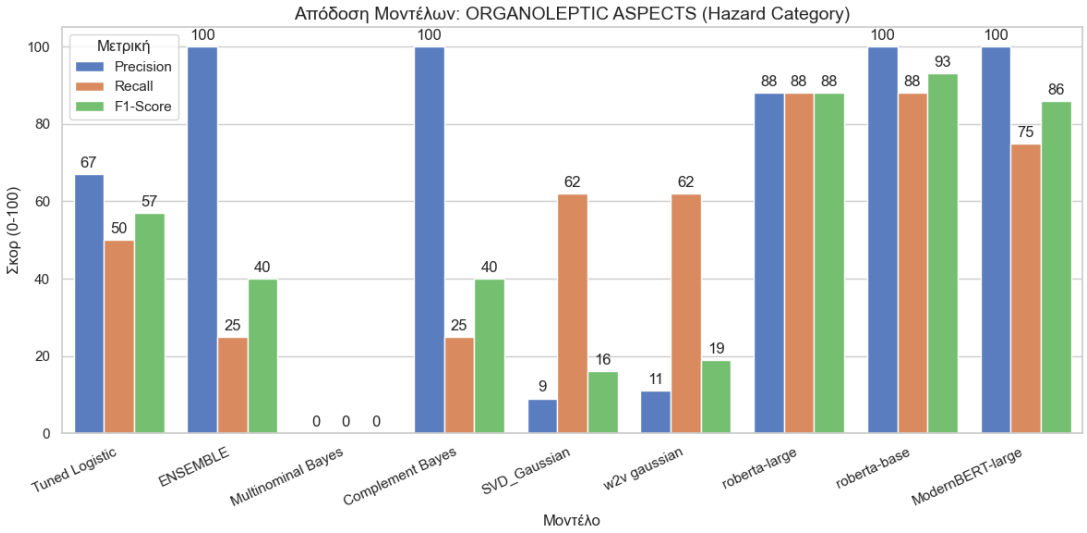
Our last metric is a confusion matrix. This matrix describes on its rows the real number of instances that belong in a category and the columns show how many instances our model predicted that belong in that category while the diagonal shows us the true positives.

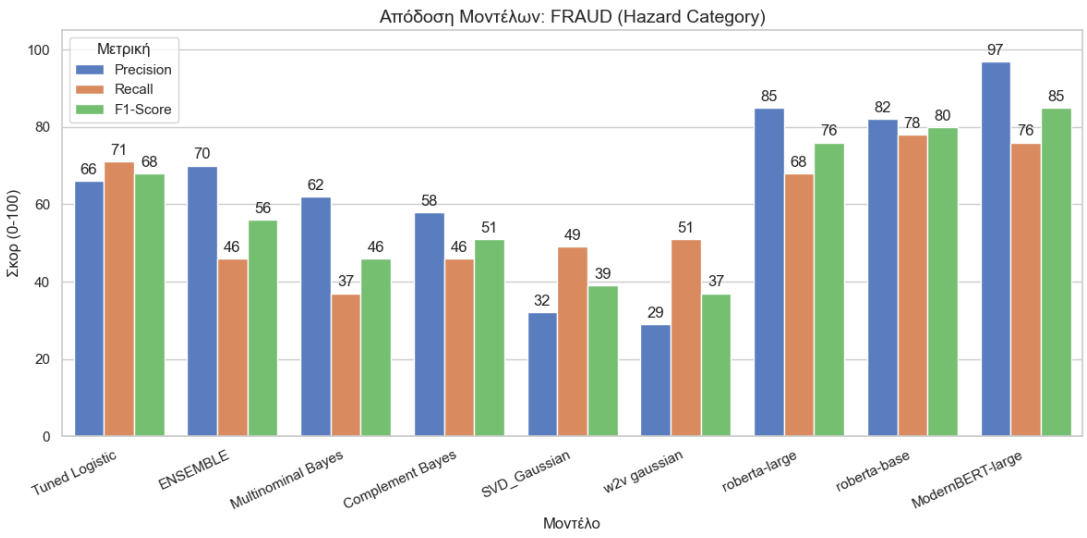
Below are listed the results of each method. Later we will discuss our observations based on the results:

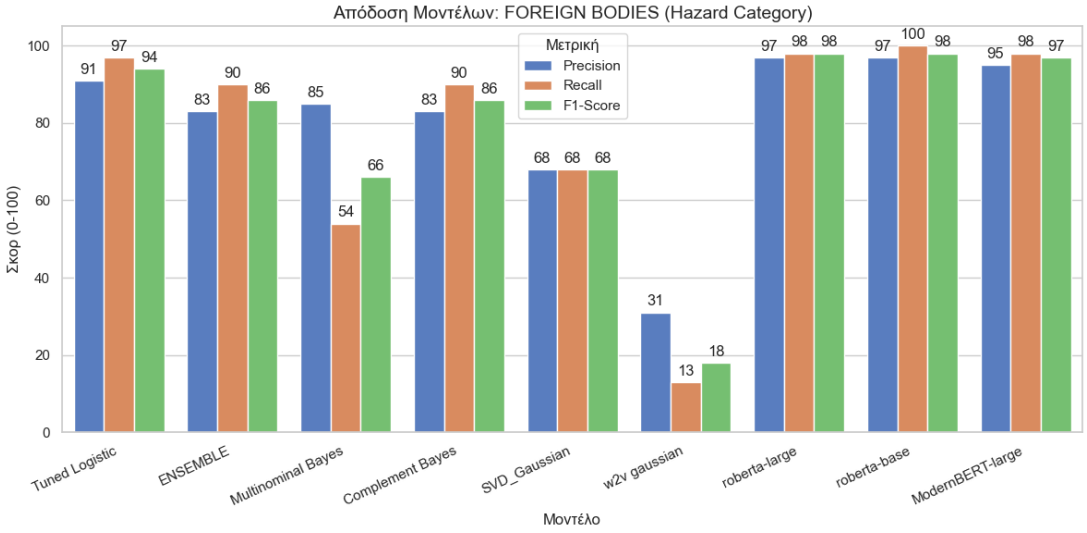
`**Hazards**

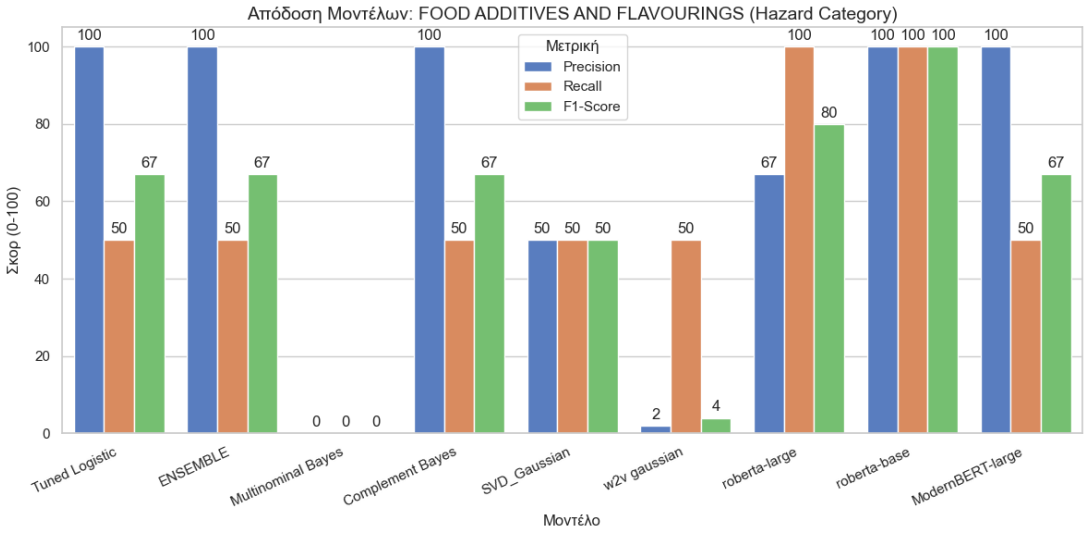

Απόδοση Μοντέλων: PACKAGING DEFECT (Hazard Category)
Μετρική
Precision
Recall
F1-Score
Σκορ (0-100)
Μοντέλο
Tuned Logistic
ENSEMBLE
Multinominal Bayes
Complement Bayes
SVD_Gaussian
w2v gaussian
roberta-large
roberta-base
ModernBERT-large
50 50 50
0 0 0
0 0 0
33 12 18
26 75 39
22 25 24
71 62 67
71 62 67
80 50 62

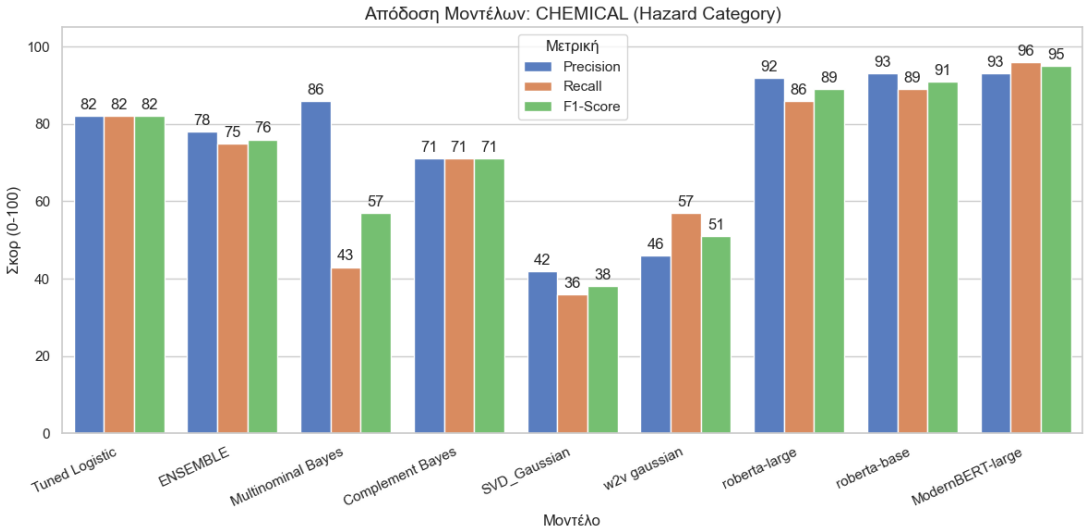

Απόδοση Μοντέλων: OTHER HAZARD (Hazard Category)
Μετρική
Precision
Recall
F1-Score
Σκορ (0-100)
Μοντέλο
Tuned Logistic
ENSEMBLE
Multinominal Bayes
Complement Bayes
SVD_Gaussian
w2v gaussian
roberta-large
roberta-base
ModernBERT-large
85 79 81
67 14 24
100 7 13
67 14 24
19 21 20
10 21 14
69 64 67
82 64 72
65 79 71

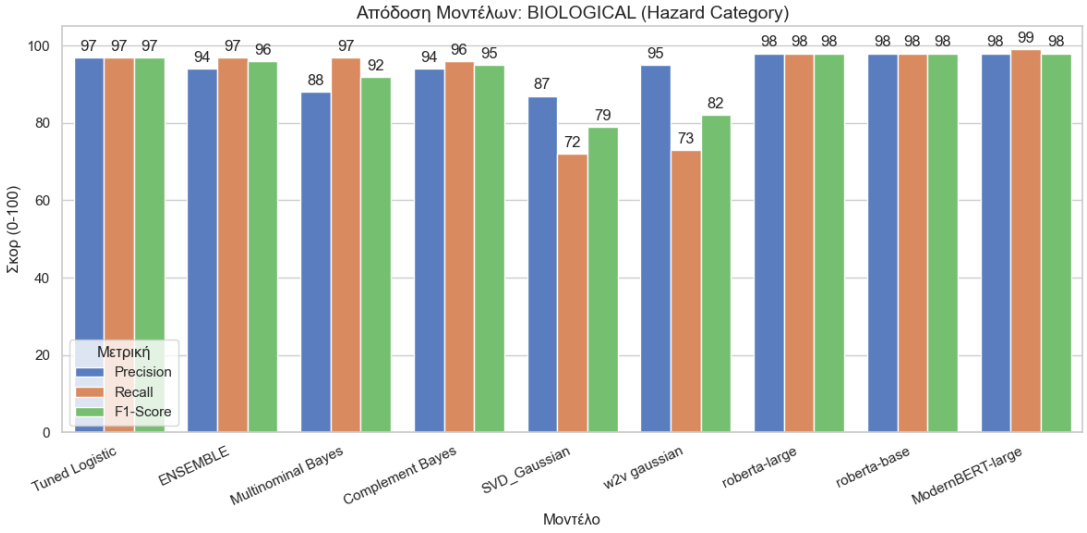

Απόδοση Μοντέλων: ORGANOLEPTIC ASPECTS (Hazard Category)
Μετρική
Precision
Recall
F1-Score
Σκορ (0-100)
Μοντέλο
Tuned Logistic
ENSEMBLE
Multinominal Bayes
Complement Bayes
SVD_Gaussian
w2v gaussian
roberta-large
roberta-base
ModernBERT-large
67 50 57
100 25 40
0 0 0
100 25 40
9 62 16
11 62 19
88 88 88
100 88 93
100 75 86

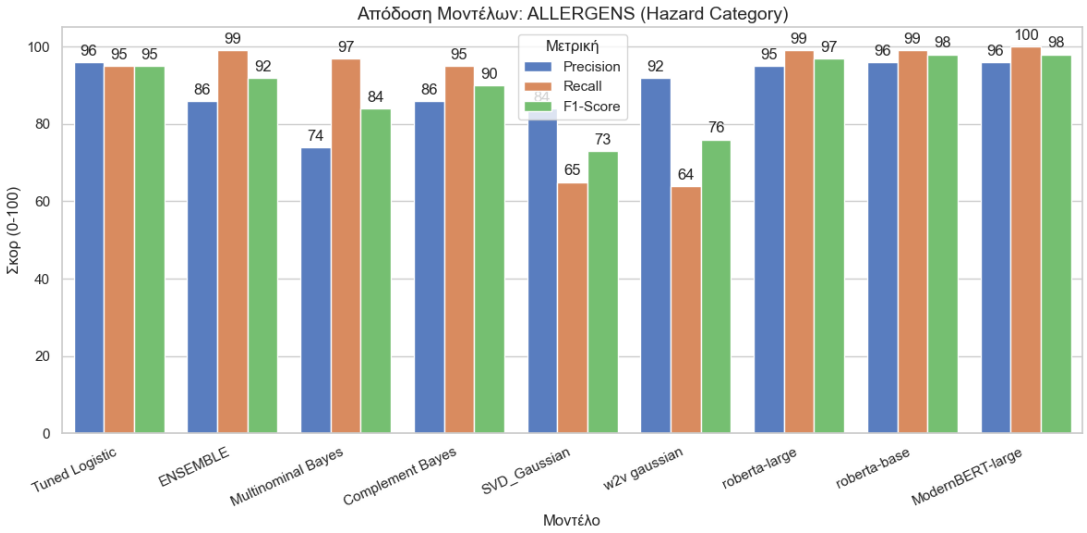

Απόδοση Μοντέλων: FRAUD (Hazard Category)
Μετρική
Precision
Recall
F1-Score
Σκορ (0-100)
Μοντέλο
Tuned Logistic
ENSEMBLE
Multinominal Bayes
Complement Bayes
SVD_Gaussian
w2v gaussian
roberta-large
roberta-base
ModernBERT-large
66 71 68
70 46 56
62 37 46
58 46 51
32 49 39
29 51 37
85 68 76
82 78 80
97 76 85


Απόδοση Μοντέλων: FOREIGN BODIES (Hazard Category)
Μετρική
Precision
Recall
F1-Score
Σκορ (0-100)
Μοντέλο
Tuned Logistic
ENSEMBLE
Multinominal Bayes
Complement Bayes
SVD_Gaussian
w2v gaussian
roberta-large
roberta-base
ModernBERT-large
91 97 94
83 90 86
85 54 66
83 90 86
68 68 68
31 13 18
97 98 98
97 100 98
95 98 97

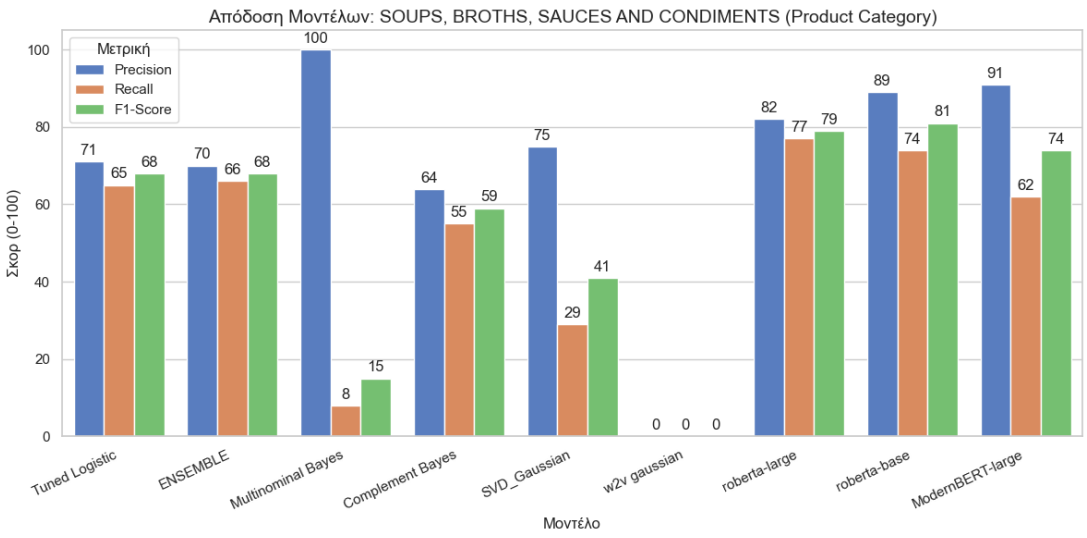

Απόδοση Μοντέλων: FOOD ADDITIVES AND FLAVOURINGS (Hazard Category)
Μετρική
Precision
Recall
F1-Score
Σκορ (0-100)
Μοντέλο
Tuned Logistic
ENSEMBLE
Multinominal Bayes
Complement Bayes
SVD_Gaussian
w2v gaussian
roberta-large
roberta-base
ModernBERT-large
100 50 67
100 50 67
0 0 0
100 50 67
50 50 50
2 50 4
67 100 80
100 100 100
100 50 67

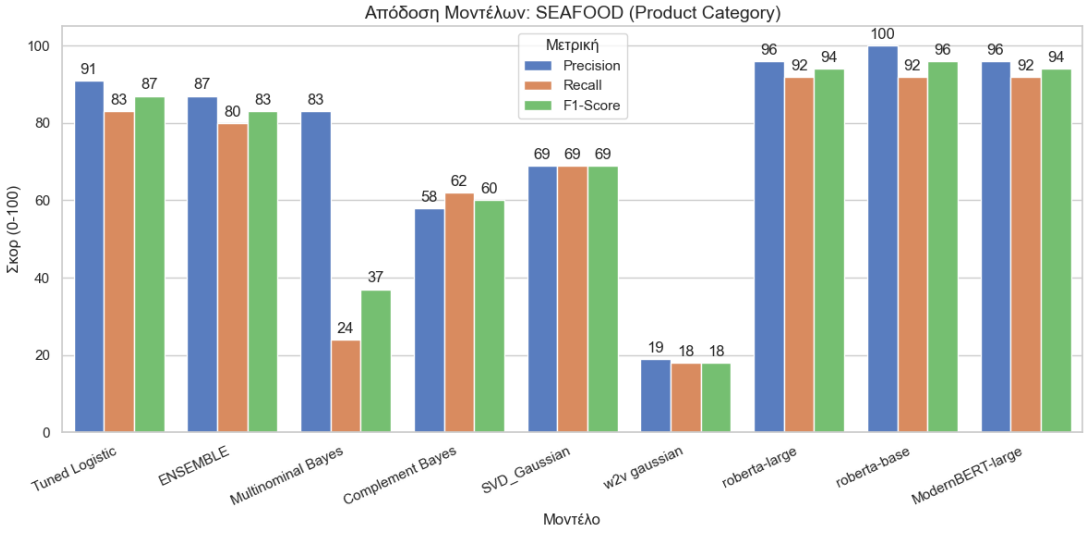

Απόδοση Μοντέλων: CHEMICAL (Hazard Category)
Μετρική
Precision
Recall
F1-Score
Σκορ (0-100)
Μοντέλο
Tuned Logistic
ENSEMBLE
Multinominal Bayes
Complement Bayes
SVD_Gaussian
w2v gaussian
roberta-large
roberta-base
ModernBERT-large
82 82 82
78 75 76
86 43 57
71 71 71
42 36 38
46 57 51
92 86 89
93 89 91
93 96 95

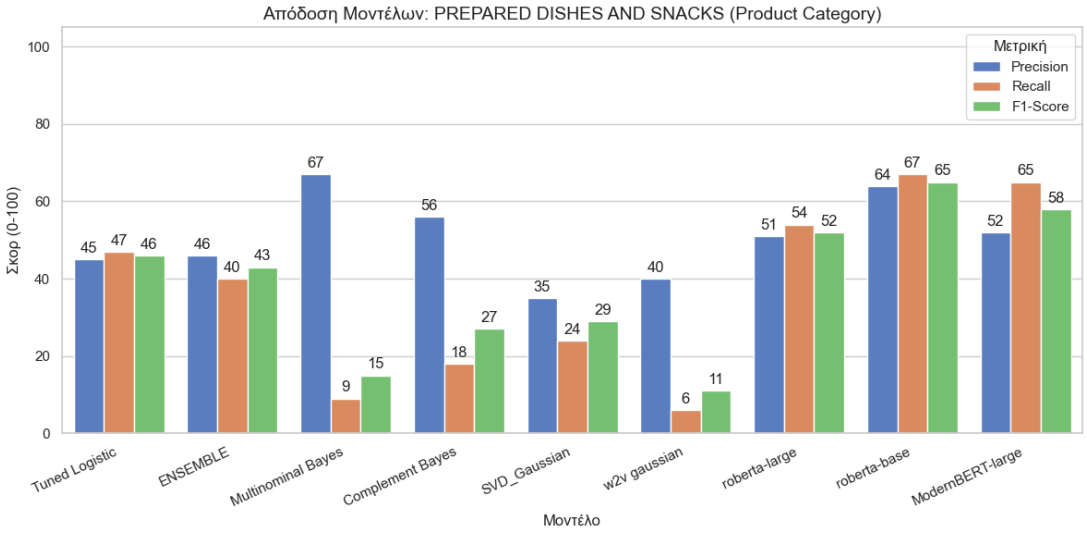

Απόδοση Μοντέλων: BIOLOGICAL (Hazard Category)
Μετρική
Precision
Recall
F1-Score
Σκορ (0-100)
Μοντέλο
Tuned Logistic
ENSEMBLE
Multinominal Bayes
Complement Bayes
SVD_Gaussian
w2v gaussian
roberta-large
roberta-base
ModernBERT-large
97 97 97
94 97 96
88 97 92
94 96 95
87 72 79
95 73 82
98 98 98
98 98 98
98 99 98

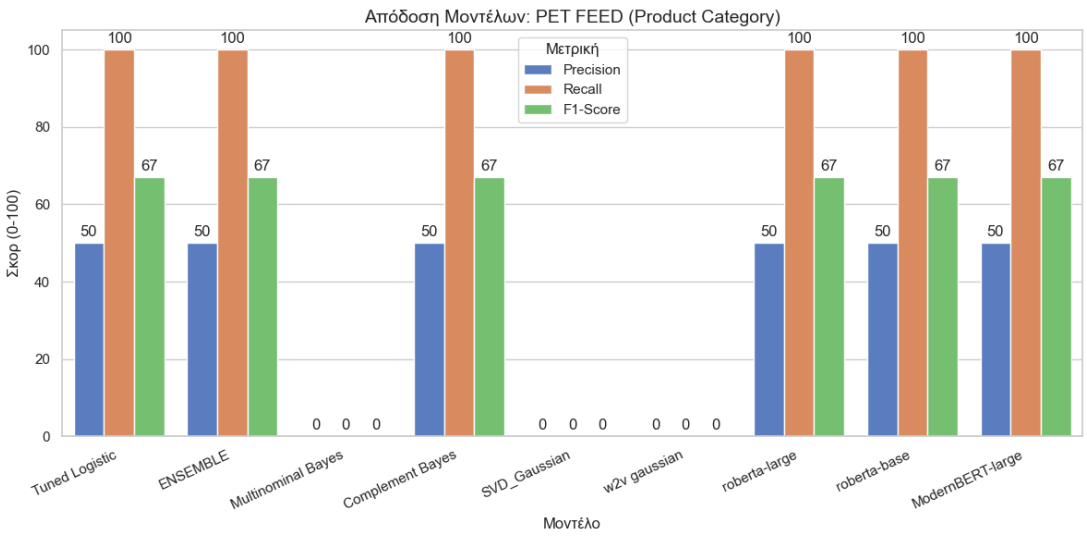

Απόδοση Μοντέλων: ALLERGENS (Hazard Category)
Μετρική
Precision
Recall
F1-Score
Σκορ (0-100)
Μοντέλο
Tuned Logistic
ENSEMBLE
Multinominal Bayes
Complement Bayes
SVD_Gaussian
w2v gaussian
roberta-large
roberta-base
ModernBERT-large
96 95 95
86 99 92
74 97 84
86 95 90
84 65 73
92 64 76
95 99 97
96 99 98
96 100 98

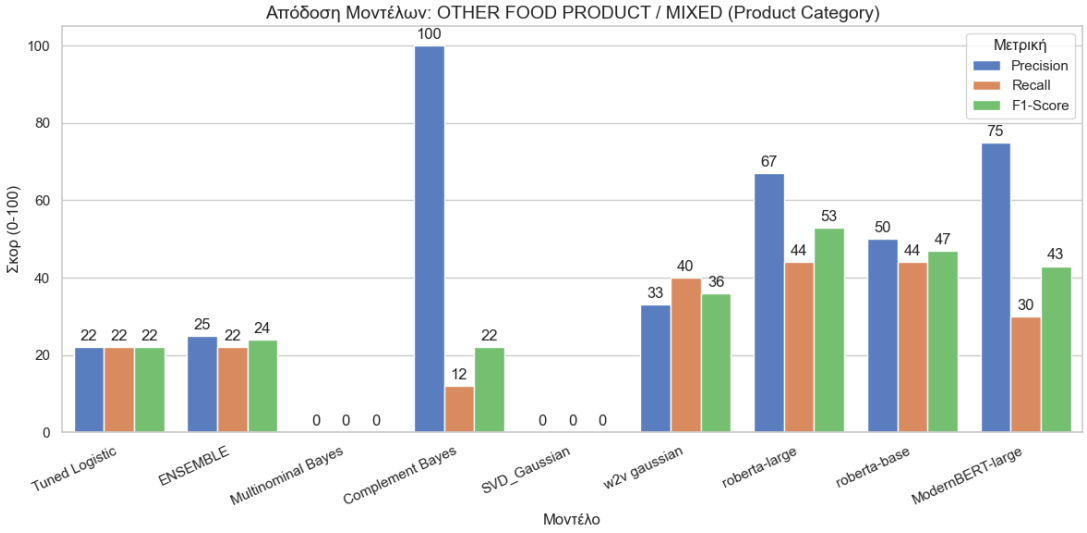
Products

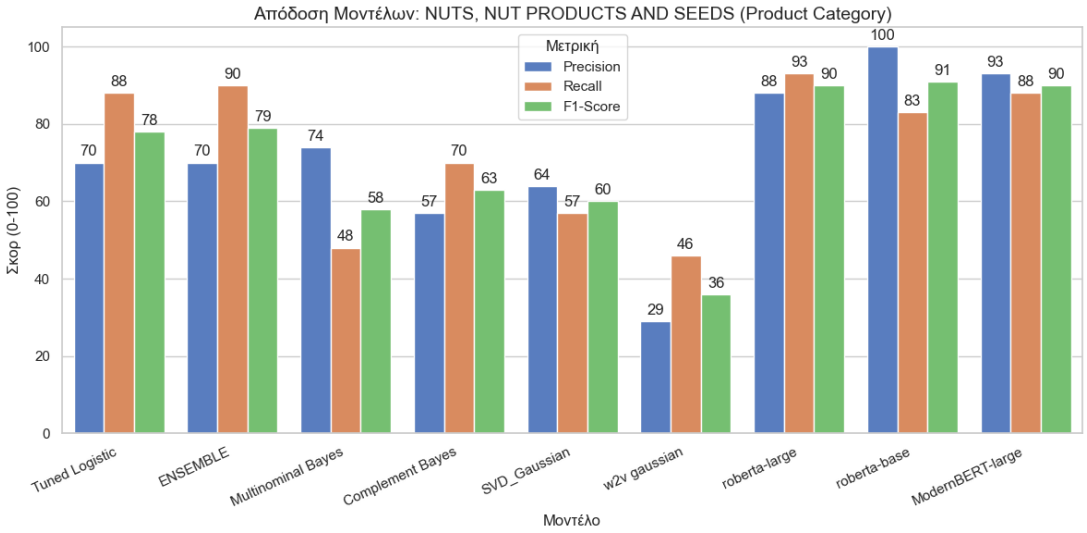

Απόδοση Μοντέλων: SOUPS, BROTHS, SAUCES AND CONDIMENTS (Product Category)
Μετρική
Precision
Recall
F1-Score
Σκορ (0-100)
Μοντέλο
Tuned Logistic
ENSEMBLE
Multinominal Bayes
Complement Bayes
SVD_Gaussian
w2v gaussian
roberta-large
roberta-base
ModernBERT-large
71 65 68
70 66 68
100 8 15
64 55 59
75 29 41
0 0 0
82 77 79
89 74 81
91 62 74

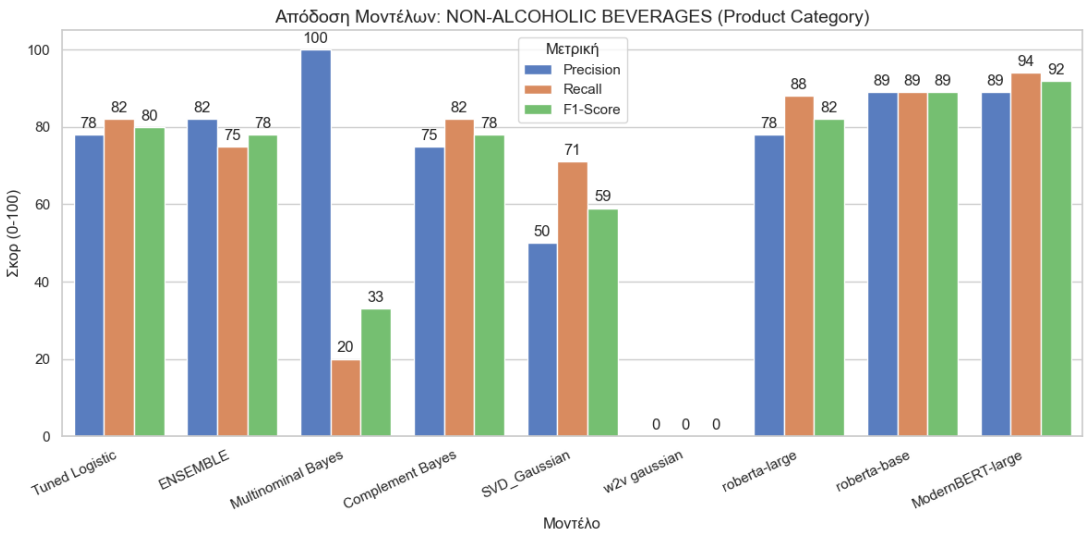

Απόδοση Μοντέλων: SEAFOOD (Product Category)
Μετρική
Precision
Recall
F1-Score
Σκορ (0-100)
Μοντέλο
Tuned Logistic
ENSEMBLE
Multinominal Bayes
Complement Bayes
SVD_Gaussian
w2v gaussian
roberta-large
roberta-base
ModernBERT-large
91 83 87
87 80 83
83 24 37
58 62 60
69 69 69
19 18 18
96 92 94
100 92 96
96 92 94

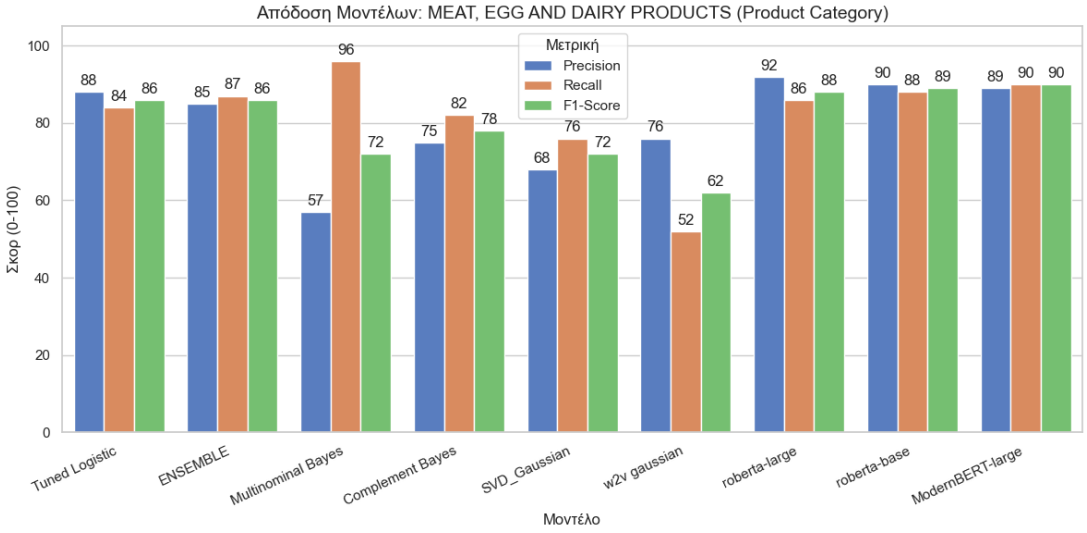

Απόδοση Μοντέλων: PREPARED DISHES AND SNACKS (Product Category)
Μετρική
Precision
Recall
F1-Score
Σκορ (0-100)
Μοντέλο
Tuned Logistic
ENSEMBLE
Multinominal Bayes
Complement Bayes
SVD_Gaussian
w2v gaussian
roberta-large
roberta-base
ModernBERT-large
45 47 46
46 40 43
67 9 15
56 18 27
35 24 29
40 6 11
51 54 52
64 67 65
52 65 58

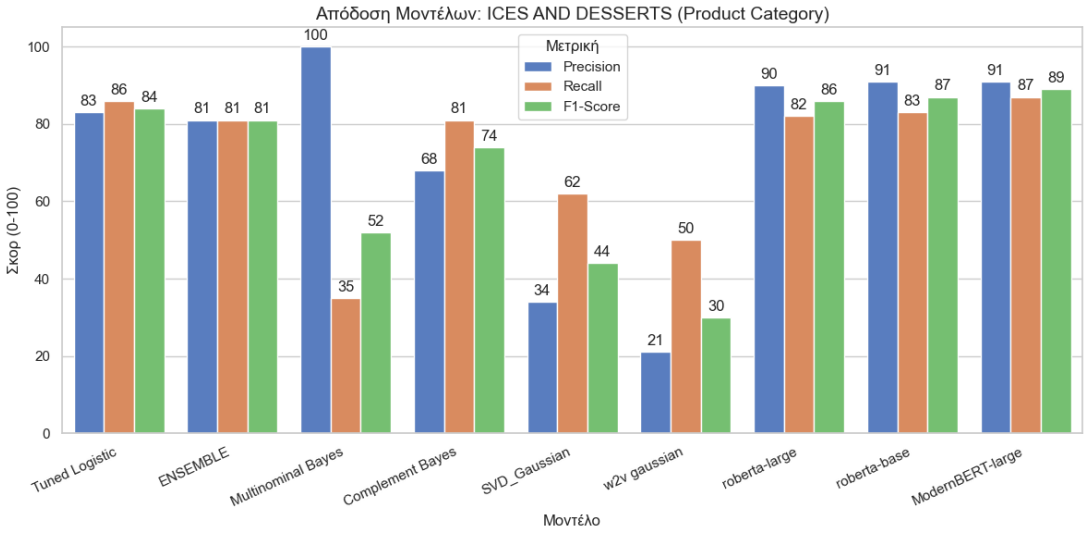

Απόδοση Μοντέλων: PET FEED (Product Category)
Μετρική
Precision
Recall
F1-Score
Σκορ (0-100)
Μοντέλο
Tuned Logistic
ENSEMBLE
Multinominal Bayes
Complement Bayes
SVD_Gaussian
w2v gaussian
roberta-large
roberta-base
ModernBERT-large

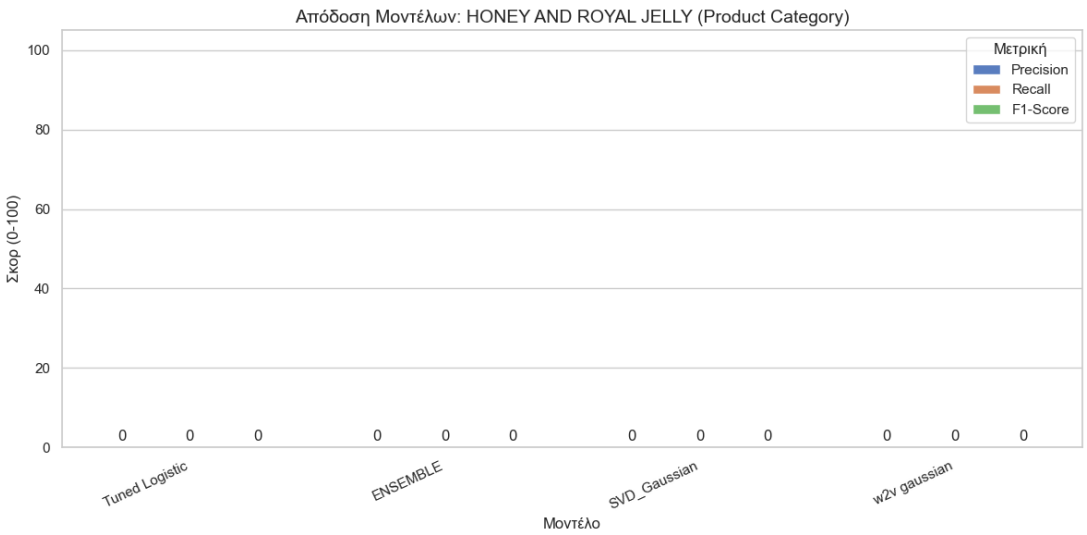

Απόδοση Μοντέλων: OTHER FOOD PRODUCT / MIXED (Product Category)
Μετρική
Precision
Recall
F1-Score
Σκορ (0-100)
Μοντέλο
Tuned Logistic
ENSEMBLE
Multinominal Bayes
Complement Bayes
SVD_Gaussian
w2v gaussian
roberta-large
roberta-base
ModernBERT-large

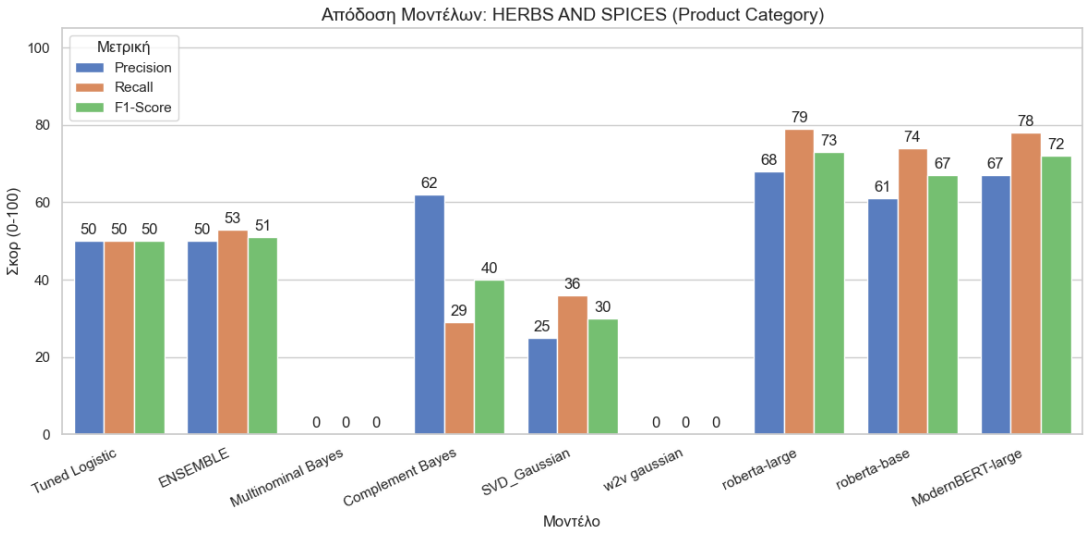

Απόδοση Μοντέλων: NUTS, NUT PRODUCTS AND SEEDS (Product Category)
Μετρική
Precision
Recall
F1-Score
Σκορ (0-100)
Μοντέλο
Tuned Logistic
ENSEMBLE
Multinominal Bayes
Complement Bayes
SVD_Gaussian
w2v gaussian
roberta-large
roberta-base
ModernBERT-large

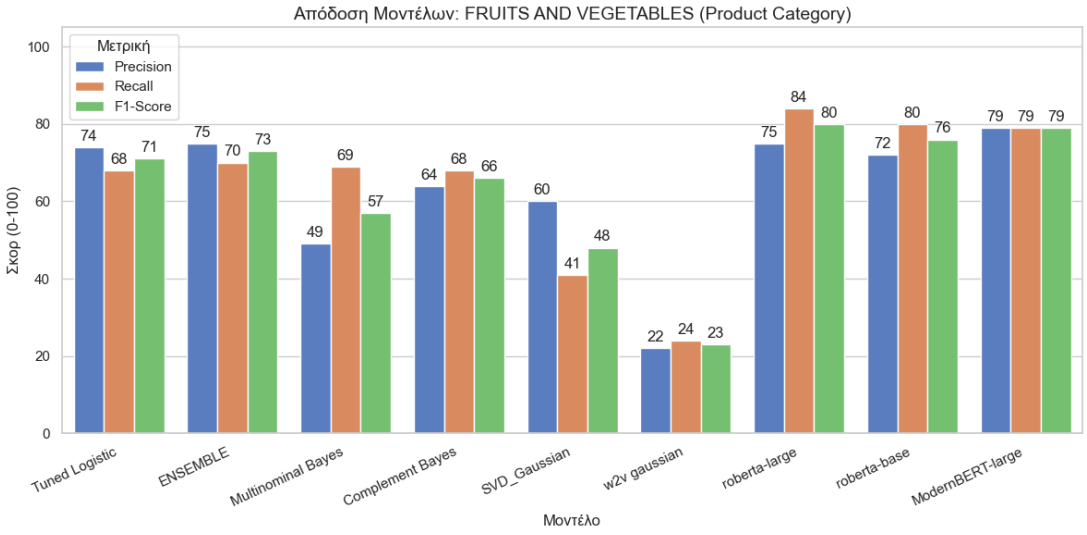

Απόδοση Μοντέλων: NON-ALCOHOLIC BEVERAGES (Product Category)
Μετρική
Precision
Recall
F1-Score
Σκορ (0-100)
Μοντέλο
Tuned Logistic
ENSEMBLE
Multinominal Bayes
Complement Bayes
SVD_Gaussian
w2v gaussian
roberta-large
roberta-base
ModernBERT-large
78 82 80
82 75 78
100 20 33
75 82 78
50 71 59
0 0 0
78 88 82
89 89 89
89 94 92

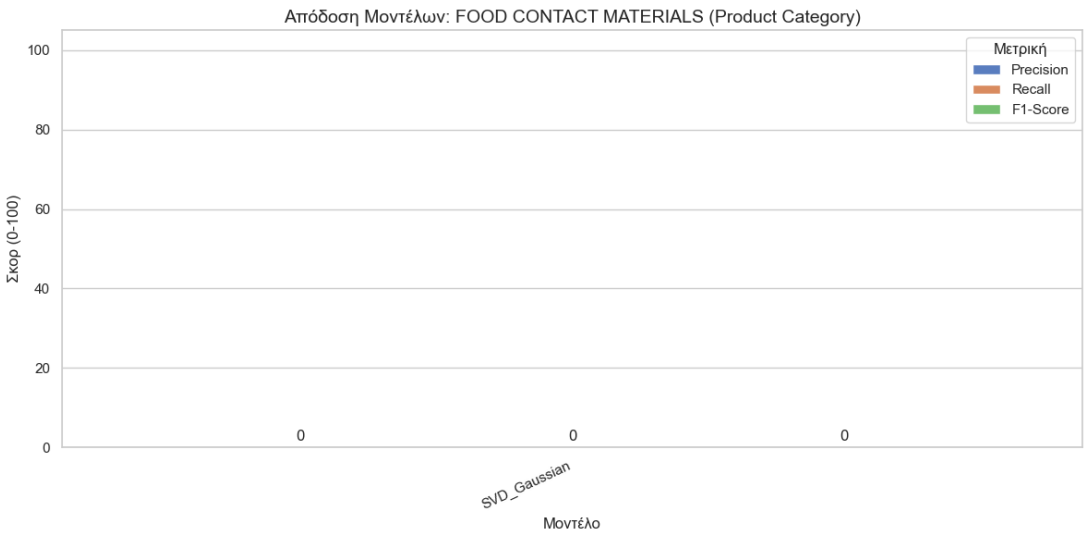

Απόδοση Μοντέλων: MEAT, EGG AND DAIRY PRODUCTS (Product Category)
Μετρική
Precision
Recall
F1-Score
Σκορ (0-100)
Μοντέλο
Tuned Logistic
ENSEMBLE
Multinominal Bayes
Complement Bayes
SVD_Gaussian
w2v gaussian
roberta-large
roberta-base
ModernBERT-large
88 84 86
85 87 86
57 96 72
75 82 78
68 76 72
76 52 62
92 86 88
90 88 89
89 90 90

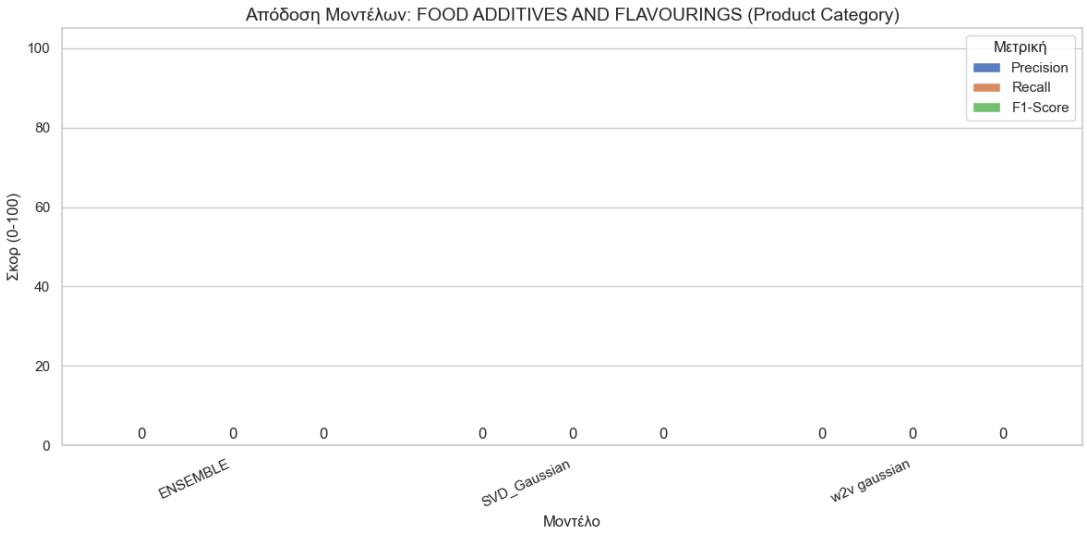

Απόδοση Μοντέλων: ICES AND DESSERTS (Product Category)
Μετρική
Precision
Recall
F1-Score
Σκορ (0-100)
Μοντέλο
Tuned Logistic
ENSEMBLE
Multinominal Bayes
Complement Bayes
SVD_Gaussian
w2v gaussian
roberta-large
roberta-base
ModernBERT-large
83 86 84
81 81 81
100 35 52
68 81 74
34 62 44
21 50 30
90 82 86
91 83 87
91 87 89

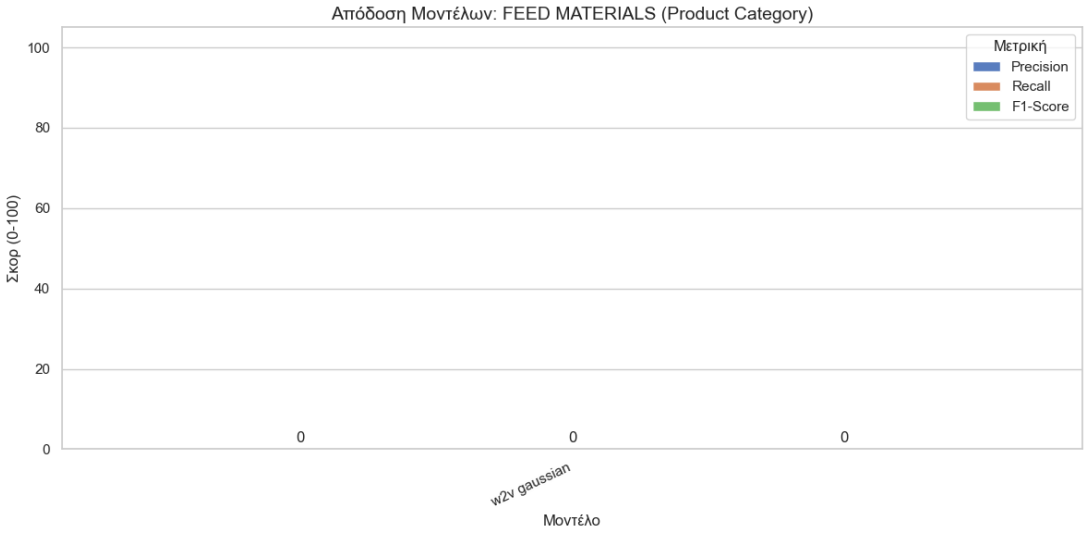

Απόδοση Μοντέλων: HONEY AND ROYAL JELLY (Product Category)
Μετρική
Precision
Recall
F1-Score
Σκορ (0-100)
Μοντέλο
Tuned Logistic
ENSEMBLE
SVD_Gaussian
w2v gaussian

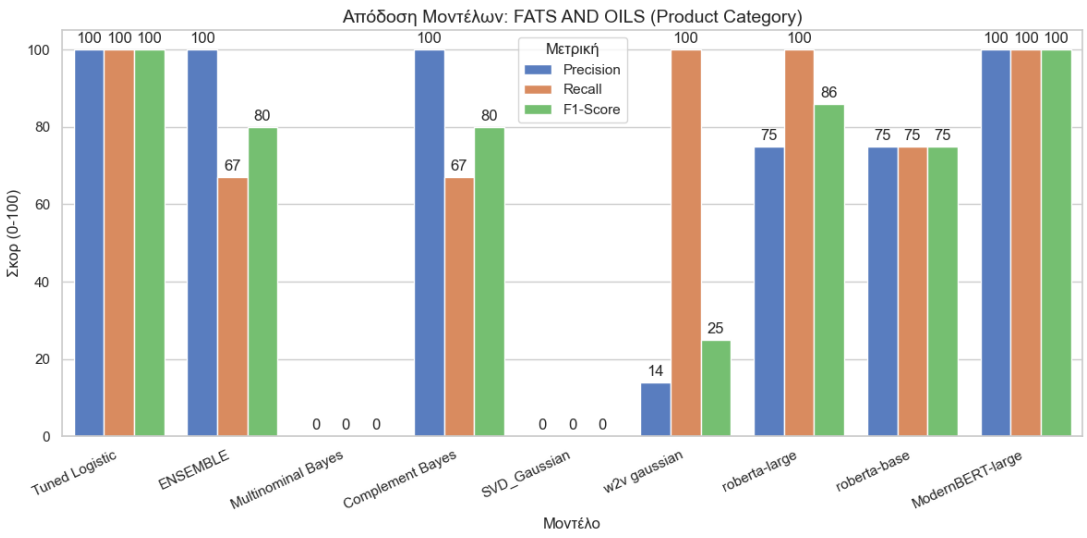

Απόδοση Μοντέλων: HERBS AND SPICES (Product Category)
Μετρική
Precision
Recall
F1-Score
Σκορ (0-100)
Μοντέλο
Tuned Logistic
ENSEMBLE
Multinominal Bayes
Complement Bayes
SVD_Gaussian
w2v gaussian
roberta-large
roberta-base
ModernBERT-large

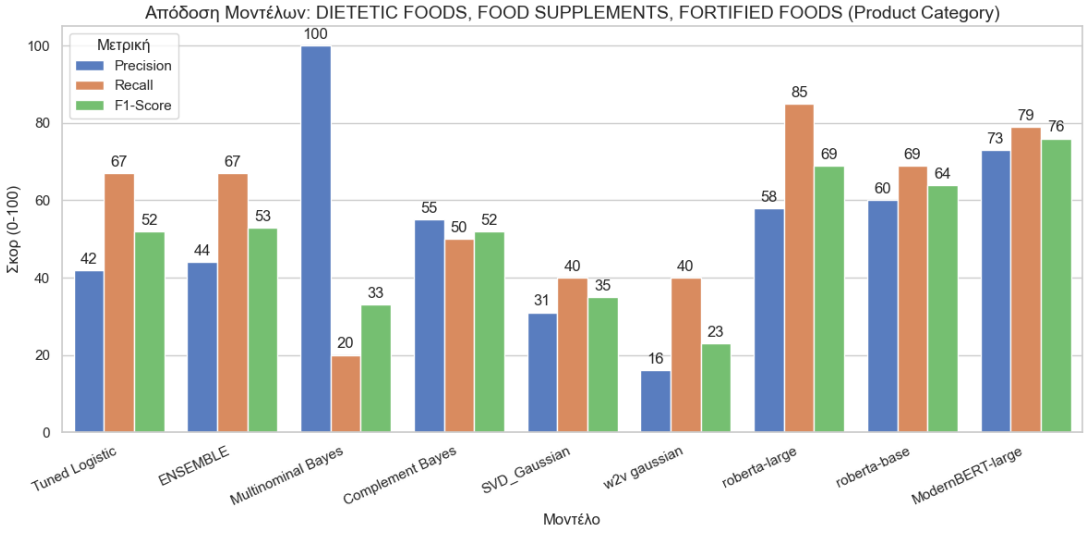

Απόδοση Μοντέλων: FRUITS AND VEGETABLES (Product Category)
Μετρική
Precision
Recall
F1-Score
Σκορ (0-100)
Μοντέλο
Tuned Logistic
ENSEMBLE
Multinominal Bayes
Complement Bayes
SVD_Gaussian
w2v gaussian
roberta-large
roberta-base
ModernBERT-large

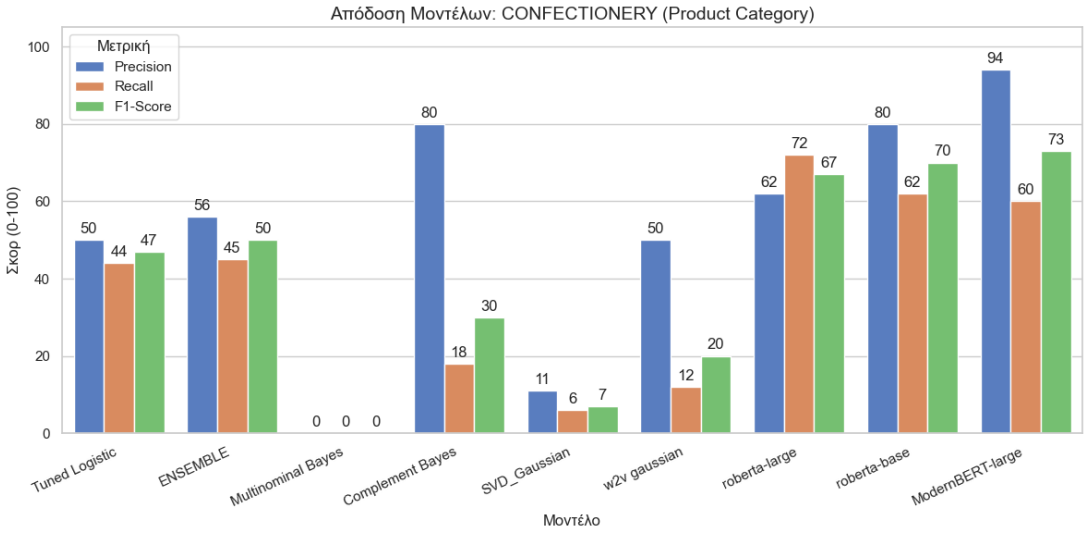

Απόδοση Μοντέλων: FOOD CONTACT MATERIALS (Product Category)
Μετρική
Precision
Recall
F1-Score
Σκορ (0-100)
100
80
60
40
20
0
0
0
0
SVD_Gaussian
Μοντέλο

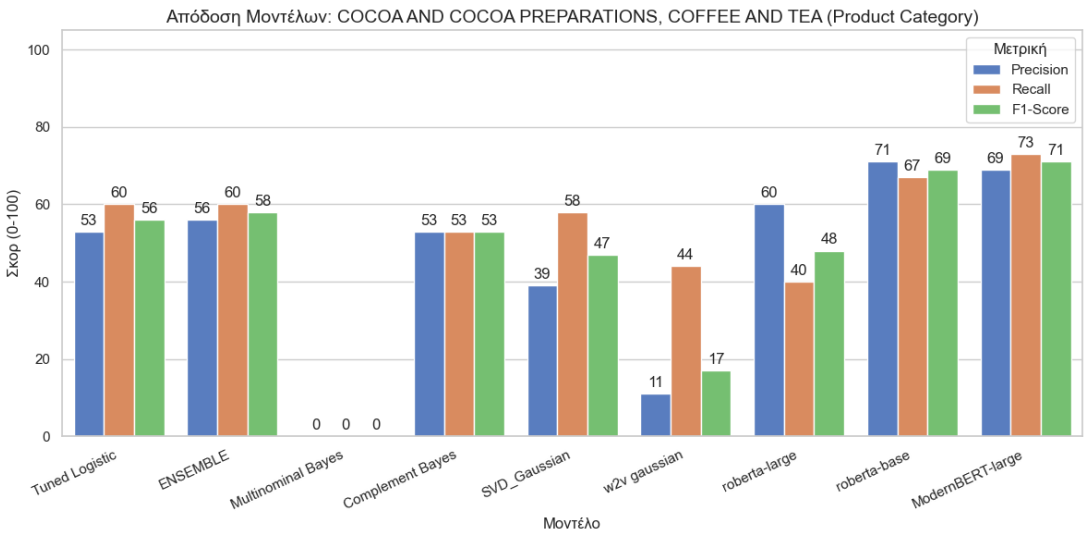

Απόδοση Μοντέλων: FOOD ADDITIVES AND FLAVOURINGS (Product Category)
Μετρική
Precision
Recall
F1-Score
Σκορ (0-100)
100
80
60
40
20
0
0 0 0 0 0 0 0 0 0
ENSEMBLE
SVD_Gaussian
w2v gaussian
Μοντέλο

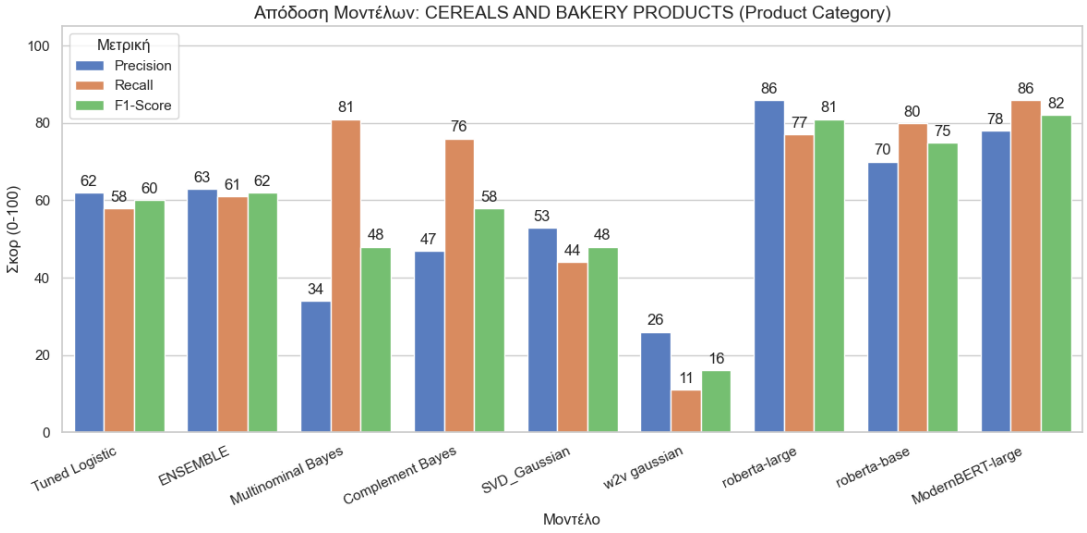

Απόδοση Μοντέλων: FEED MATERIALS (Product Category)
Μετρική
Precision
Recall
F1-Score
Σκορ (0-100)
100
80
60
40
20
0
0
0
0
w2v gaussian
Μοντέλο

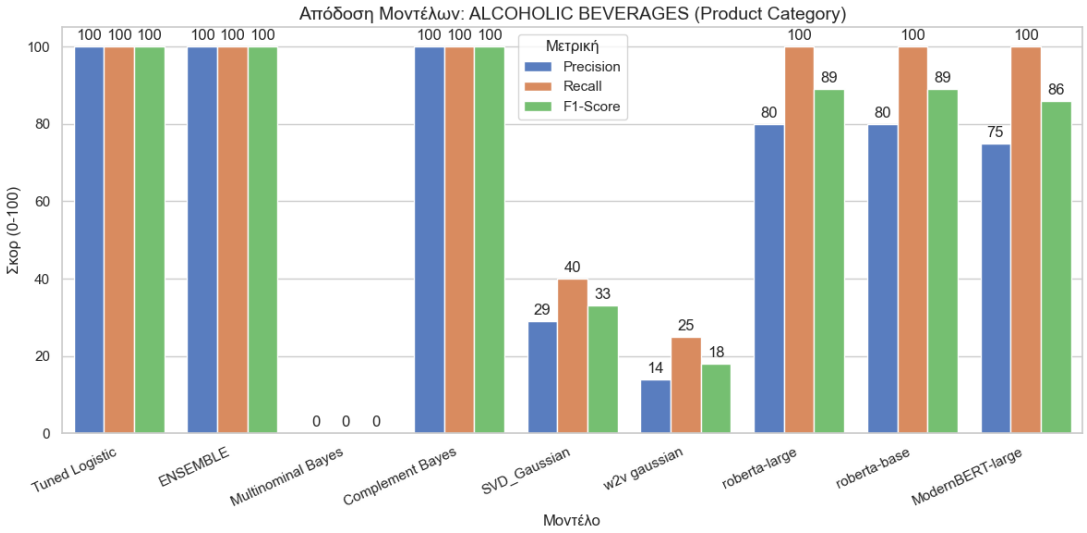

Απόδοση Μοντέλων: FATS AND OILS (Product Category)
Μετρική
Precision
Recall
F1-Score
Σκορ (0-100)
Μοντέλο
Tuned Logistic
ENSEMBLE
Multinominal Bayes
Complement Bayes
SVD_Gaussian
w2v gaussian
roberta-large
roberta-base
ModernBERT-large


Απόδοση Μοντέλων: DIETETIC FOODS, FOOD SUPPLEMENTS, FORTIFIED FOODS (Product Category)
Μετρική
Precision
Recall
F1-Score
Σκορ (0-100)
Μοντέλο
Tuned Logistic
ENSEMBLE
Multinominal Bayes
Complement Bayes
SVD_Gaussian
w2v gaussian
roberta-large
roberta-base
ModernBERT-large

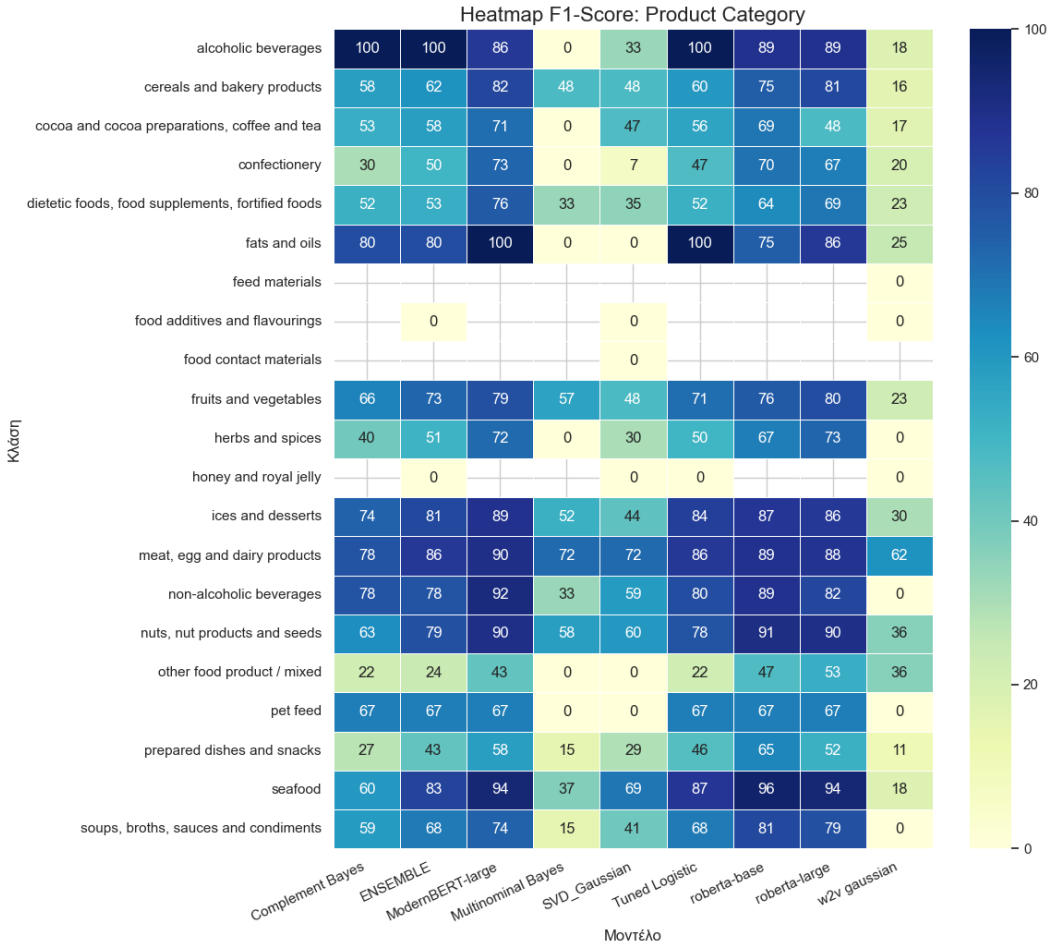

Απόδοση Μοντέλων: CONFECTIONERY (Product Category)
Μετρική
Precision
Recall
F1-Score
Σκορ (0-100)
Μοντέλο
Tuned Logistic
ENSEMBLE
Multinominal Bayes
Complement Bayes
SVD_Gaussian
w2v gaussian
roberta-large
roberta-base
ModernBERT-large

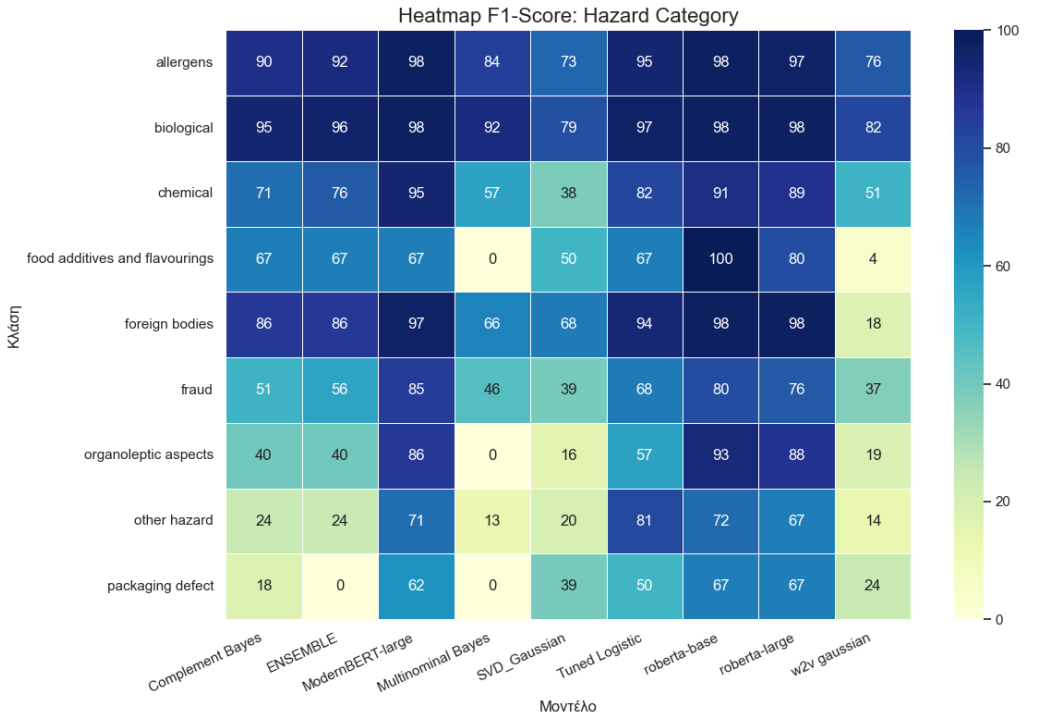

Απόδοση Μοντέλων: COCOA AND COCOA PREPARATIONS, COFFEE AND TEA (Product Category)
Μετρική
Precision
Recall
F1-Score
Σκορ (0-100)
Μοντέλο
Tuned Logistic
ENSEMBLE
Multinominal Bayes
Complement Bayes
SVD_Gaussian
w2v gaussian
roberta-large
roberta-base
ModernBERT-large
53 60 56
56 60 58
0 0 0
53 53 53
39 58 47
11 44 17
60 40 48
71 67 69
69 73 71


Απόδοση Μοντέλων: CEREALS AND BAKERY PRODUCTS (Product Category)
Μετρική
Precision
Recall
F1-Score
Σκορ (0-100)
Μοντέλο
Tuned Logistic
ENSEMBLE
Multinominal Bayes
Complement Bayes
SVD_Gaussian
w2v gaussian
roberta-large
roberta-base
ModernBERT-large
62 58 60
63 61 62
34 81 48
47 76 58
53 44 48
26 11 16
86 77 81
70 80 75
78 86 82

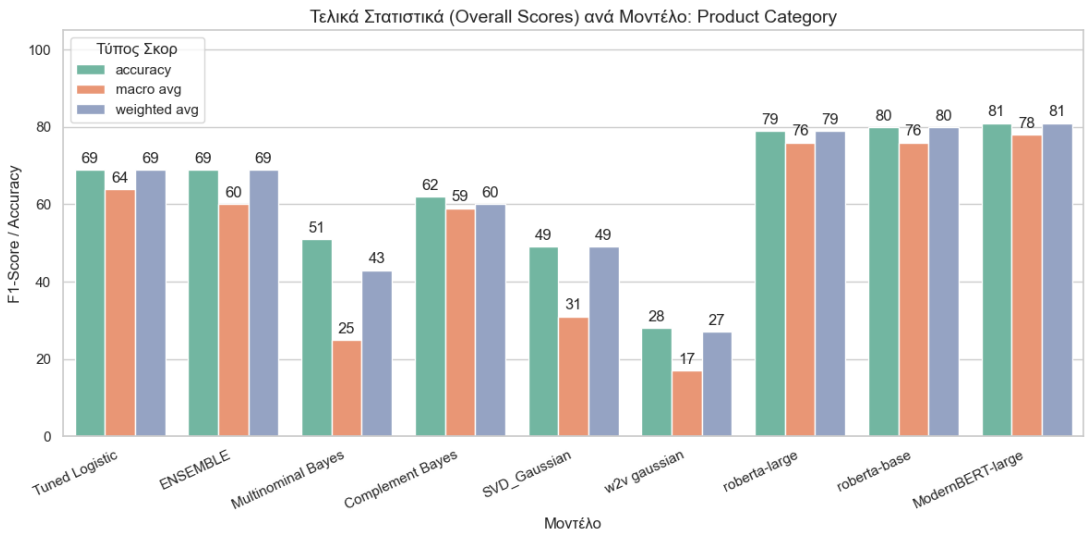

Απόδοση Μοντέλων: ALCOHOLIC BEVERAGES (Product Category)
Μετρική
Precision
Recall
F1-Score
Σκορ (0-100)
Μοντέλο
Tuned Logistic
ENSEMBLE
Multinominal Bayes
Complement Bayes
SVD_Gaussian
w2v gaussian
roberta-large
roberta-base
ModernBERT-large
100 100 100
100 100 100
0 0 0
100 100 100
29 40 33
14 25 18
80 100 89
80 100 89
75 100 86

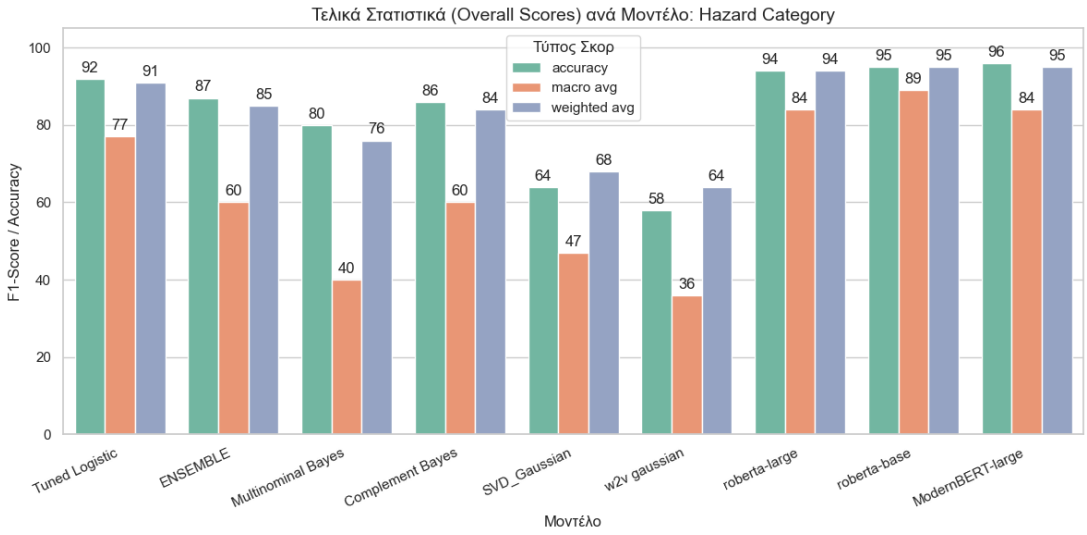
Heatmaps


Heatmap F1-Score: Product Category
alcoholic beverages 100 100 86 0 33 100 89 89 18
cereals and bakery products 58 62 82 48 48 60 75 81 16
cocoa and cocoa preparations, coffee and tea 53 58 71 0 47 56 69 48 17
confectionery 30 50 73 0 7 47 70 67 20
dietetic foods, food supplements, fortified foods 52 53 76 33 35 52 64 69 23
fats and oils 80 80 100 0 0 100 75 86 25
feed materials 0
food additives and flavourings 0 0 0
food contact materials 0
fruits and vegetables 66 73 79 57 48 71 76 80 23
herbs and spices 40 51 72 0 30 50 67 73 0
honey and royal jelly 0 0 0 0
ices and desserts 74 81 89 52 44 84 87 86 30
meat, egg and dairy products 78 86 90 72 72 86 89 88 62
non-alcoholic beverages 78 78 92 33 59 80 89 82 0
nuts, nut products and seeds 63 79 90 58 60 78 91 90 36
other food product / mixed 22 24 43 0 0 22 47 53 36
pet feed 67 67 67 0 0 67 67 67 0
prepared dishes and snacks 27 43 58 15 29 46 65 52 11
seafood 60 83 94 37 69 87 96 94 18
soups, broths, sauces and condiments 59 68 74 15 41 68 81 79 0
Κλάση
Complement Bayes
ENSEMBLE
ModernBERT-large
Multinominal Bayes
SVD_Gaussian
Tuned Logistic
roberta-base
roberta-large
w2v gaussian
Μοντέλο
100
80
60
40
20
0


Heatmap F1-Score: Hazard Category
allergens 90 92 98 84 73 95 98 97 76
biological 95 96 98 92 79 97 98 98 82
chemical 71 76 95 57 38 82 91 89 51
food additives and flavourings 67 67 67 0 50 67 100 80 4
foreign bodies 86 86 97 66 68 94 98 98 18
fraud 51 56 85 46 39 68 80 76 37
organoleptic aspects 40 40 86 0 16 57 93 88 19
other hazard 24 24 71 13 20 81 72 67 14
packaging defect 18 0 62 0 39 50 67 67 24
Κλάση
Complement Bayes
ENSEMBLE
ModernBERT-large
Multinominal Bayes
SVD_Gaussian
Tuned Logistic
roberta-base
roberta-large
w2v gaussian
Μοντέλο
100
80
60
40
20
0

Overall





Comments About The Charts

From the overall charts we can have a clear picture of the results we got from each method. The worst were the w2v gaussian as well as the svd implementation meaning that the model couldn't create a good enough vector similarity. After that comes the multinominal Bayes which is out scaled massively by the Complement Bayes due to the imbalance of the data. From the baselines methods we can see that the tuned logistic and the ensembled tuned logistic with complement bayes gave a solid result but all of them were outperformed from the "state-of-the-art" transformers. From the charts we can see that all three gave decent and almost identical f1 scores for the hazard category, but Roberta-base gave a little better result for the product category. Finally, the heatmaps revealed the imbalance on the categories that hurt the f1 score (packaging defect, other hazard, other food hazard / mixed etc.) which is helpful determining where our model struggled to predict correctly.

Overall, our opinion is that the logistic regression and complement Bayes gave solid results for the tasks as baseline methods without needing heavy computational power or effort making them work. On the other hand, transformers gave a lot stronger scores on the validation test making it possible to find the rare categories. The downside is that some models require heavy computational power (models were ran in A100 40GB gpu) making sure we can cover the whole text and not lose the meaning. They also require a considerable amount of data to have good predictability, hence the use of data augmentation for categories with low sample. Finally, they need some work achieving the suitable parameters so they can work effectively (token length, batch size, epochs, learning rate).